# LogicLab — Phased Build Plan for AI Agent

Oct 26, 2026 · @Mobeen

## How to use this doc

Send the agent one phase at a time, always together with the Project Overview and Database Schema sections. Wait until a phase passes all its acceptance checks before sending the next one.

1. First message to the agent: paste **Project Overview** + **Database Schema** + **Phase 0**.
2. Each next message: paste the next phase and say: *"Continue the same project. Follow the global rules. Do not rewrite finished phases unless this phase requires it."*
3. After each phase, test the acceptance checks yourself in the browser before moving on.
4. If something breaks, send the agent the exact error message and the phase number, not a vague description.

Phases 0–3 are the MVP: after them, students can sign up, use the trial, pay with a slip, and get an invoice. Phases 4–11 add the learning features one by one.

## Project Overview (send with every phase)

Build LogicLab, a paid tech-learning web app for students in Pakistan with three tracks: Logic Building, Web Development, and AI/ML & Data Science. It has three portals (Student, Mentor, Admin) plus a CMS, a 7-day free trial, and manual payment verification via EasyPaisa, JazzCash, or bank slip upload.

### Tech stack (free tiers only)

| Layer | Choice |
|---|---|
| Framework | Next.js 15 (App Router), TypeScript strict mode |
| UI | Tailwind CSS + shadcn/ui, lucide-react icons |
| Database, auth, realtime | Supabase (Postgres, Supabase Auth, Realtime, Row Level Security) |

| Layer | Choice |
|---|---|
| File storage | Cloudflare R2 via S3-compatible API, presigned URLs only |
| Hosting | Cloudflare Workers using the OpenNext Cloudflare adapter ( `@opennextjs/cloudflare` ) |
| Validation | Zod on every form and server action |
| Email | Resend (invoice and notification emails) |
| PDF invoices | `@react-pdf/renderer` or `pdf-lib` |
| OCR for slips | Tesseract.js (client side) |
| Rate limiting | Upstash Redis |
| Live classes | Jitsi Meet embed |
| Code editor | Monaco Editor; Pyodide for Python in the browser |
| AI | Google Gemini API or Groq, behind one `lib/ai.ts` wrapper so the provider can be swapped |
| Errors | Sentry free tier |

## Folder structure

```
/app
  /(public)        landing, pricing, about, login, signup
  /(student)       /student/... dashboard, courses, quizzes, community,
billing
  /(mentor)        /mentor/... courses, grading, chat, live classes
  /(admin)         /admin/... CMS, users, payments, analytics, audit
  /api             webhooks and route handlers only
/components        shared UI (ui/ from shadcn)
/lib               supabase clients, r2, ai, email, pdf, auth helpers,
validators
/supabase/migrations   all SQL, one file per change
/types             generated Supabase types
```

## Global rules for the agent

- Use TypeScript everywhere; no `any` .

- Every table has Row Level Security enabled with explicit policies. Never use the service-role key in client code.
- Role checks happen in three places: middleware (route groups), server actions, and RLS policies.
- All database changes go in `/supabase/migrations` as SQL files; never edit the database by hand.
- All secrets live in `.env.local`; keep `.env.example` updated with every new variable.
- Files never go into Supabase Storage; use R2 with short-lived presigned URLs.
- Mobile-first, responsive, light and dark mode. Professional look: clean spacing, one accent color, no clutter.
- UI text in English, but keep all strings in `/lib/i18n` so Urdu can be added later.
- Every admin action that changes money, roles, or bans writes a row to `audit_logs`.
- Do not use any paid service. If a feature seems to need one, stop and ask.
- At the end of each phase, list: files created, migrations added, env variables added, and how to test.

## Database Schema (send with every phase)

These are the core tables. Each phase creates only the tables it needs, in its own migration. All tables use `uuid` primary keys, `created_at`, and `updated_at`.

| Area | Tables | Created in |
|---|---|---|
| Users & auth | `profiles` (id = auth.users.id, full_name, avatar_url, role enum: student/mentor/admin/super_admin, status: active/banned), `user_sessions` (device limit), `audit_logs` | Phase 1 |
| Courses | `tracks`, `courses`, `modules`, `lessons`, `course_mentors`, `site_content` (CMS blocks for landing page) | Phase 2 |
| Billing | `plans`, `subscriptions` (status: trial/pending/active/expired), `payments` (method, transaction_id UNIQUE, amount, slip_key, ocr_text, status: pending/approved/rejected, reviewed_by, reject_reason), `invoices` (invoice_no, pdf_key), `coupons` | Phase 3 |

| Area | Tables | Created in |
|---|---|---|
| Learning | `enrollments`, `lesson_progress`, `lesson_questions`, `lesson_answers` | Phase 4 |
| Assessments | `question_bank`, `quizzes`, `quiz_questions`, `quiz_attempts`, `assignments`, `submissions` (content, typing_events jsonb, flags jsonb, grade, feedback), `rubrics` | Phase 5 |
| Community | `posts`, `post_answers`, `votes`, `reports` | Phase 6 |
| Realtime & points | `conversations`, `messages`, `point_events`, `seasons`, `notifications` | Phase 7 |
| Live classes | `live_sessions`, `attendance` | Phase 8 |
| AI | `ai_usage` (per user, per day, for rate limits) | Phase 9 |
| Practice | `challenges`, `challenge_submissions` | Phase 10 |
| Rewards | `season_winners`, `certificates` (verify_code), `mentor_ratings` | Phase 11 |

Key rules: a student sees only their own `payments`, `submissions`, and `messages`; a mentor sees submissions only for courses in `course_mentors`; admins see everything; `audit_logs` is insert-only and admin-read-only.

## Phase 0: Project Setup

**Goal:** an empty but deployable app with the stack, folders, and design system ready.

**Tasks for the agent:**

1. Create the Next.js 15 app with TypeScript, Tailwind, ESLint, Prettier, and shadcn/ui.
2. Set up the folder structure from the Project Overview.
3. Add Supabase server and browser clients in `/lib/supabase`, plus type generation script.
4. Add the R2 client in `/lib/r2.ts` with helpers: `getUploadUrl(key, contentType)` and `getDownloadUrl(key)`, both expiring in 10 minutes.
5. Configure `@opennextjs/cloudflare` with `wrangler.toml` and a `deploy` script.

6. Build the base layout: public navbar, footer, theme toggle, and empty dashboard shells for `/student`, `/mentor`, `/admin` with sidebars.
7. Create the public landing page branded "LogicLab" (logo text, site title, and meta tags use the name) with sections: hero, the 3 tracks, how it works, pricing preview, FAQ, and footer.
8. Add Sentry, `.env.example`, and a GitHub Actions workflow that runs lint, type-check, and deploy on push to `main`.

**Acceptance checks:**

- [ ] `npm run build` passes with no type errors.
- [ ] Landing page looks professional on mobile and desktop, in light and dark mode.
- [ ] Deploy to Cloudflare works and the site opens on the `workers.dev` URL.

## Phase 1: Authentication and Roles

**Goal:** secure login with Google or verified email, and each role sees only its own portal.

**Tasks for the agent:**

1. Migration: `profiles`, `user_sessions`, `audit_logs`, role and status enums. Add a trigger that creates a `profiles` row (role = student) on every new auth user.
2. Supabase Auth with two methods: Google OAuth, and email + password with email OTP verification. An account cannot log in until its email is verified.
3. Pages: `/login`, `/signup`, `/verify-email`, `/forgot-password`, `/reset-password`.
4. Middleware: unauthenticated users are redirected to `/login`; `/student` allows student; `/mentor` allows mentor; `/admin` allows admin and super_admin. Banned users are logged out with a message.
5. Helper `requireRole(roles[])` for server actions; never trust the role sent from the client.
6. RLS policies for `profiles`: users read and update only their own row, except `role` and `status`, which only admins can change.
7. Two-factor authentication (TOTP via Supabase MFA) required for mentor and admin accounts before they can open their portal.
8. Device limit: max 2 active sessions per student; logging in on a third device signs out the oldest session.

9. Rate limit login and OTP endpoints with Upstash (5 attempts per 15 minutes per IP and per email).
10. Seed script that creates one super_admin from env variables.

**Acceptance checks:**

- [ ] Google login and email signup both work; unverified email cannot log in.
- [ ] A student opening `/admin` is redirected and blocked.
- [ ] A student cannot change their own role, even with a direct API call.
- [ ] Mentor and admin are forced to set up 2FA.
- [ ] Third device login signs out the oldest session.

## Phase 2: CMS and Course Structure

**Goal:** admin builds tracks, courses, and lessons, and edits the landing page without code.

**Tasks for the agent:**

1. Migration: `tracks`, `courses` (level: beginner/intermediate/advanced, is_published), `modules` (order, unlock_after_module_id), `lessons` (type: video/article/quiz/assignment, order), `course_mentors`, `site_content`.
2. Seed the three tracks: Logic Building, Web Development, AI/ML & Data Science.
3. Admin CMS at `/admin/content`: create, edit, reorder (drag and drop), publish, and unpublish tracks, courses, modules, lessons.
4. Rich text editor for article lessons (Tiptap), with code blocks and images uploaded to R2.
5. Admin can assign mentors to courses.
6. Admin user management at `/admin/users`: search, filter by role, change role, ban/unban, with reason. Every change writes to `audit_logs`.
7. `site_content` editor: hero text, FAQ items, testimonials, banners. The landing page reads from this table.
8. Public course catalog `/courses` and course detail page showing modules and lesson titles (content locked).

**Acceptance checks:**

- [ ] Admin creates a course with modules and lessons, and it appears in the public catalog after publishing.
- [ ] Changing hero text in the CMS updates the landing page.

- [ ] Role change and ban appear in the audit log.

## Phase 3: Free Trial, Payments, and Invoices

**Goal:** 7-day trial, then payment by slip upload, admin approval, and an emailed PDF invoice.

**Tasks for the agent:**

1. Migration: `plans` (name, price_pkr, duration_days, track_ids), `subscriptions`, `payments` with UNIQUE `transaction_id`, `invoices`, `coupons`.
2. On first login, create a `trial` subscription for 7 days, limited to one chosen track. One trial per verified email and per Google account.
3. Access check helper `hasAccess(userId, courseId)` used by pages and RLS; expired users see a paywall with a link to `/student/billing`.
4. Billing page: choose plan, apply coupon, see payment instructions (EasyPaisa number, JazzCash number, bank IBAN, all editable in admin settings).
5. Slip upload form: method, transaction ID, amount, date, and slip image (JPG/PNG/PDF, max 5 MB) uploaded to R2 via presigned URL.
6. Run Tesseract.js on the slip in the browser; save extracted text, and show a warning to the admin if the amount or transaction ID is not found in it.
7. Block duplicate transaction IDs with a clear error message.
8. Admin queue `/admin/payments`: slip preview, OCR text, student info, Approve / Reject (reason required).
9. On approve, in one database transaction: activate subscription, create invoice number (format `INV-2026-00001`), generate PDF invoice, save to R2, email it via Resend, write audit log.
10. On reject, email the student the reason and let them re-upload.
11. Student billing history with invoice downloads; reminder emails 3 days and 1 day before expiry (Supabase scheduled function or Cloudflare Cron Trigger).

**Acceptance checks:**

- [ ] New student gets 7 days on one track, then sees the paywall.
- [ ] Same transaction ID cannot be submitted twice.
- [ ] Approval activates access and the invoice PDF arrives by email.
- [ ] A student cannot see another student's slip or invoice.

# Phase 4: Recorded Lectures and Progress

**Goal:** mentors upload videos; students watch, ask questions under lessons, and track progress.

**Tasks for the agent:**

1. Migration: `enrollments`, `lesson_progress` (watched_seconds, completed), `lesson_questions` (timestamp_seconds), `lesson_answers`.
2. Mentor upload page: upload video to R2 with progress bar (multipart for big files), plus notes and attachments.
3. Video player with hls.js (or plain MP4 if HLS conversion is not set up yet), using short-lived signed URLs, speed control, and resume from last position. Add a light watermark with the student's email to discourage screen recording.
4. Lesson page: video or article, notes, downloadable resources, and a timestamped Q&A tab (click a timestamp to jump to that point).
5. Mark lesson complete at 90% watched; next module unlocks only when the previous module's quiz is passed (quiz built in Phase 5; until then, unlock on completion).
6. Student dashboard: continue learning card, completion % per course, streak counter, and time spent this week.

**Acceptance checks:**

- [ ] Video URLs expire and cannot be shared.
- [ ] Student resumes the video where they left off.
- [ ] Questions under a lesson show the timestamp and mentor answers.

# Phase 5: Assignments and Quizzes with Integrity Checks

**Goal:** mentors create timed quizzes and assignments; students answer in a write-only editor; mentors grade.

**Tasks for the agent:**

1. Migration: `question_bank` (type: mcq/short/code, difficulty, tags), `quizzes` (time_limit, starts_at, ends_at, pass_mark), `quiz_questions`, `quiz_attempts`, `assignments` (due_at, max_marks), `submissions`, `rubrics`.
2. Mentor pages: question bank CRUD, build a quiz from the bank, create assignment with rubric, schedule start and end time.

3. Quiz player: one question at a time, server-side timer (never trust the client clock), auto-submit at time end, MCQs auto-graded on the server.
4. Write-only answer editor for assignments and short answers:
   - Block paste, drop, and right-click paste; show a small notice when paste is attempted.
   - Record typing events (timestamp, chars added) in batches every 10 seconds into `typing_events`.
   - Flag a submission if more than 200 characters appear within 1 second, or if typing speed is unrealistic.
   - Save drafts automatically every 15 seconds.
5. Tab-switch detection during quizzes: count `visibilitychange` events and save them on the attempt.
6. Mentor grading page: submission text, flags, typing replay (play back how the answer was typed), rubric scoring, written feedback, return to student.
7. Student sees grade, feedback, and correct answers after the quiz closes.
8. Hook module unlocking (from Phase 4) to quiz pass marks.

**Acceptance checks:**

- [ ] Pasting into the answer box does nothing.
- [ ] A quiz auto-submits when time ends, even if the student closes the tab and reopens it.
- [ ] Mentor can replay how an answer was typed and sees integrity flags.
- [ ] A mentor can only grade submissions for their assigned courses.

## Phase 6: Community Q&A Feed

**Goal:** students and mentors post questions like posts; anyone answers; best answers get points.

**Tasks for the agent:**

1. Migration: `posts` (title, body, code_snippet, tags, track_id, accepted_answer_id), `post_answers`, `votes` (unique per user per item), `reports`.
2. Feed page `/community`: filter by track and tag, sort by newest, most voted, unanswered; search by title.
3. Post editor with markdown and code blocks with syntax highlighting; images to R2.
4. Answers with upvote/downvote; the asker or a mentor marks one answer as accepted.

5. Mentor answers show a "Mentor" badge.
6. Report button on posts and answers; admin moderation queue at `/admin/moderation` to hide, delete, or warn the user.
7. Mentor announcements: a pinned post type for tips and "new in tech" updates.

**Acceptance checks:**

- [ ] A user can vote only once per post or answer.
- [ ] Only the asker or a mentor can accept an answer.
- [ ] Reported content appears in the admin queue.

## Phase 7: Realtime Chat, Points, Leaderboard, and Notifications

**Goal:** students chat with mentors live, earn points for learning, and get notified.

**Tasks for the agent:**

1. Migration: `conversations` (student_id, mentor_id, course_id, status: open/waiting/closed), `messages`, `point_events` (user_id, reason, points, source_id), `seasons` (monthly), `notifications`.
2. Chat with Supabase Realtime: student opens a doubt from a course; it goes into the mentor's queue; mentor picks it up. Support code blocks and image attachments, typing indicator, and read receipts.
3. Mentor office hours: mentors set available times; outside them, the chat shows "mentor will reply during office hours".
4. Points engine as one server function `awardPoints(userId, reason, sourceId)`, never called from the client. Default rules (admin can edit): quiz pass +20, assignment submitted on time +15, accepted answer +10, 7-day streak +25, live class attended +5, plagiarism flag confirmed -50.
5. Leaderboard per season and per track, updated live; resets each month.
6. Notifications: in-app bell (realtime), plus email for important events (grade returned, payment approved, live class in 30 minutes). Users can turn email types on or off.

**Acceptance checks:**

- [ ] Student and mentor messages appear instantly without refresh.
- [ ] Points cannot be added from the browser console.
- [ ] Leaderboard shows the current month only and updates live.

# Phase 8: Live Classes

**Goal:** mentors schedule live classes; students join inside the app; attendance is tracked.

**Tasks for the agent:**

1. Migration: `live_sessions` (course_id, mentor_id, title, starts_at, duration, room_name, recording_key), `attendance` (joined_at, left_at).
2. Mentor schedules a session; enrolled students see it in a calendar on their dashboard and get reminders (Phase 7 notifications).
3. Embed Jitsi Meet using the Jitsi iframe API with a random, unguessable room name. The join button appears only 10 minutes before start and only for students with access.
4. Record attendance when a student joins and leaves; award points for attending.
5. After the class, the mentor uploads the recording (from Jitsi or their own screen recorder) to R2, and it appears as a lesson.

**Acceptance checks:**

- [ ] Students without access cannot get the room link.
- [ ] Attendance shows join and leave times in the mentor's view.

# Phase 9: AI Features

**Goal:** an AI tutor for students and AI helpers for mentors, within free API limits.

**Tasks for the agent:**

1. `lib/ai.ts`: one wrapper `askAI({ system, messages })` for Gemini or Groq, chosen by env variable. All calls happen on the server only.
2. Migration: `ai_usage`; limit each student to 30 AI messages per day (admin can change); show remaining count.
3. AI tutor panel inside each lesson: the system prompt includes the lesson content and tells the AI to give hints and explanations, never full assignment answers.
4. AI code reviewer: student clicks "Review my code" on a practice exercise (not on graded assignments before submission).
5. Mentor helpers: generate quiz questions from a lesson (mentor reviews before saving to the question bank), and suggested feedback on a submission based on the rubric (mentor edits and approves).
6. AI similarity check after submission: flag answers that look AI-written or very similar to another student's answer; add to submission flags for the mentor, never auto-penalize.

7. Recommendations on the dashboard: weak topics from quiz results with suggested lessons.

**Acceptance checks:**

- [ ] API key never appears in the browser network tab.
- [ ] Daily limit stops requests with a friendly message.
- [ ] AI tutor refuses to write the full answer to an active assignment.

## Phase 10: Code Editor and Daily Challenges

**Goal:** students write and run code in the browser, and solve one challenge every day.

**Tasks for the agent:**

1. Monaco Editor component with JavaScript and Python modes; paste blocking reused from Phase 5 when used for graded work.
2. Run JavaScript in a sandboxed Web Worker with a 5-second timeout; run Python with Pyodide in a Web Worker (load on demand, cache it). Include numpy and pandas for the AI/ML track.
3. Migration: `challenges` (track, difficulty, statement, starter_code, test_cases jsonb, hidden_tests jsonb, publish_date), `challenge_submissions`.
4. Test runner: visible tests run in the browser; for graded challenges, re-run hidden tests in a server-side check before saving the result (Phase 10 keeps it simple; Judge0 on a VPS can replace this later).
5. Daily challenge page: one per track per day, streak tracking, points on first correct solve.
6. Notebook-style page for the AI/ML track: cells that run Python with Pyodide and show tables and charts.

**Acceptance checks:**

- [ ] An infinite loop is stopped after 5 seconds without freezing the page.
- [ ] Python with pandas runs in the browser.
- [ ] Points are awarded only once per challenge.

# Phase 11: Rewards, Certificates, Portfolio, Analytics, and Launch

**Goal:** reward top students, issue verifiable certificates, give admins analytics, and harden the app for launch.

**Tasks for the agent:**

1. Migration: `season_winners` (season_id, user_id, rank, reward_type: fee_refund/cash/free_month, amount_pkr, paid_at), `certificates` (verify_code), `mentor_ratings`.
2. Admin competition page: close a season, review the top 10 (with integrity flags shown), confirm winners, record payouts. Reward budget is a fixed number set by admin, never calculated from other students' fees. Trial users are not eligible.
3. Public rules page for the competition, editable in the CMS.
4. Certificate PDF on track completion with a QR code linking to `/verify/[code]`, a public page that confirms the certificate is real.
5. Public student portfolio `/u/[username]`: projects, badges, certificates; student chooses what is public.
6. Mentor ratings after each course; shown to admins only.
7. Admin analytics dashboard: signups per day, trial-to-paid conversion %, active students, revenue per month (from approved payments), churn, top courses, mentor response time.
8. Launch hardening: security headers, RLS review of every table, check all server actions for `requireRole`, image and file type validation, SEO meta tags and sitemap, privacy policy and terms pages, PWA manifest for mobile install.

**Acceptance checks:**

- [ ] Scanning a certificate QR opens the verify page with the correct name and track.
- [ ] Analytics numbers match the raw data in the database.
- [ ] A full RLS test script confirms students, mentors, and admins can only access what they should.

Before launching the cash competition, confirm the prize rules with a lawyer in Pakistan.